

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – 3

Roll No.: T017	Name: Radhika Mali
Class: TYIT	Batch: 1
Date of Assignment: 14/07/2026	Date/Time of Submission: 14/07/2026

a) Create methods add(), multiply(), subtract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

Step 1: Start the program.

Step 2: Create a delegate and define calculator methods (Add, Sub, Mul, Div).

Step 3: Create an object of the Calculator class.

Step 4: Assign each method to the delegate and invoke it.

Step 5: Display the results and stop the program.

Step 6: Stop.

Code:

```
using System;
delegate void Operation(int a, int b);
class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition = " + (a + b));
    }
    public void Sub(int a, int b)
    {
        Console.WriteLine("subtraction = " + (a - b));
    }
    public void Mul(int a, int b)
    {
        Console.WriteLine("Multiplication = " + (a * b));
    }
    public void Div(int a, int b)
    {
        Console.WriteLine("Division = " + (a / b));
    }
}
class Program
```

Radhika 17

```
{
    static void Main()
    {
        Calculator c = new Calculator();
        Operation obj;
        obj = c.Add;
        obj(1, 2);
        obj = c.Sub;
        obj(2, 2);
        obj = c.Mul;
        obj(4, 2);
        obj = c.Div;
        obj(6, 2);
        Console.WriteLine("Radhika 17");
    }
}
```

Output:

```
Addition = 3
subtraction = 0
Multiplication = 8
Division = 3
Radhika 17
```

b) Write a program using multicast delegates.

Algorithm:

Step 1: Start the program.

Step 2: Create a delegate and define the methods Welcome(), Learn(), and Thanks().

Step 3: Create an object of the Demo class.

Step 4: Add all methods to the delegate and invoke it.

Step 5: Display the messages and stop the program.

Step 6: Stop

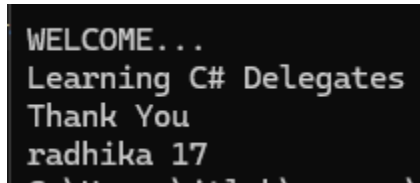
Code:

```
using System;
delegate void Message();
class Demo
{
    public void Welcome()
    {
        Console.WriteLine("WELCOME...");
    }
    public void Learn()
    {
        Console.WriteLine("Learning C# Delegates");
    }
}
```

Radhika 17

```
}  
public void Thanks()  
{  
    Console.WriteLine("Thank You");  
}  
}  
public class Program  
{  
    public static void Main(string[] args)  
    {  
        Demo d = new Demo();  
        Message m;  
  
        m = d.Welcome;  
        m += d.Learn;  
        m += d.Thanks;  
        m();  
  
        Console.Write("radhika 17");  
  
    }  
}
```

Output:

A screenshot of a terminal window showing the output of the program. The text is displayed in a monospaced font on a dark background. The output consists of four lines: "WELCOME...", "Learning C# Delegates", "Thank You", and "radhika 17".

```
WELCOME...  
Learning C# Delegates  
Thank You  
radhika 17
```

c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

Step 1: Start the program.

Step 2: Create the BankAccount class with AccountNumber and Balance properties.

Step 3: Create an object of the BankAccount class.

Step 4: Assign and display the account number and balance.

Step 5: Update the balance, display the details, and stop the program.

Step 6: Stop

Code:

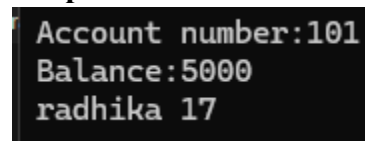
```
class BankAccount  
{  
    public int AccountNumber { get; set; }  
    private double balance;  
    public double Balance  
    {
```

Radhika 17

```
    get
    {
        return balance;
    }
    set
    {
        if (balance >= 0)
            balance = value;
        else
            Console.WriteLine("balance cannot be negative");
    }
}

class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();
        b.AccountNumber = 101;
        b.Balance = 5000;
        Console.WriteLine("Account number:" + b.AccountNumber);
        Console.WriteLine("Balance:" + b.Balance);
        b.Balance = -1000;
        Console.WriteLine("radhika 17");
    }
}
```

Output:

A screenshot of a terminal window showing the output of the program. The text is displayed on three lines: "Account number:101", "Balance:5000", and "radhika 17". The background is black and the text is white.

```
Account number:101
Balance:5000
radhika 17
```